

Registering new User:

The screenshot shows a REST client interface with the following details:

- URL:** `127.0.0.1:8000/docs#/Authentication/register_auth_register_post`
- Buttons:** "Execute" (blue) and "Clear" (white).
- Responses:** A section titled "Responses" is visible.
- Curl:**

```
curl -X 'POST' \
  'http://127.0.0.1:8000/auth/register' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "Alina Akhtar",
    "password": "dumypass12"
  }'
```
- Request URL:** `http://127.0.0.1:8000/auth/register`
- Server response:**

Code	Details
200	Response body <pre>{ "id": 3, "username": "Alina Akhtar" }</pre> Download
- Response headers:** A section for response headers is visible at the bottom.

Logging in as User:

The screenshot shows a REST client interface with the following details:

- URL:** `127.0.0.1:8000/docs#/Authentication/login_auth_login_post`
- Form Fields:**
 - `string | (string | null)` with a text input containing "string" and a "Send empty value" checkbox.
 - `client_secret` with a text input containing "*****" and a "Send empty value" checkbox.
- Buttons:** "Execute" (blue) and "Clear" (white).
- Responses:** A section titled "Responses" is visible.
- Curl:**

```
curl -X 'POST' \
  'http://127.0.0.1:8000/auth/login' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  -d 'grant_type=password&username=Alina%20Akhtar&password=dumypass12&scope=&client_id=string&client_secret=*****'
```
- Request URL:** `http://127.0.0.1:8000/auth/login`
- Server response:**

Code	Details
200	Response body <pre>{ "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjozLCJleHAiOjE3ODQwMzg2MDZ9.7WwMb8_QgdPjL9i441G6n_zxSjXtbbeKw6jJcxdBc7Y", "token_type": "bearer" }</pre> Download

Without the unauthorized access:

The screenshot shows a web browser window with the address bar displaying `127.0.0.1:8000/docs#/Tasks/create_task_tasks_post`. The interface includes a top bar with "Execute" and "Clear" buttons. Below this, the "Responses" section is visible. The "Curl" section contains the following command:

```
curl -X 'POST' \
  'http://127.0.0.1:8000/tasks/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "title": "string",
    "description": "string",
    "status": "pending",
    "due_date": "2026-07-14"
  }'
```

The "Request URL" section shows `http://127.0.0.1:8000/tasks/`. The "Server response" section displays a 401 status code with the message "Error: Unauthorized". The "Response body" section shows the following JSON:

```
{
  "detail": "Not authenticated"
}
```

The "Response headers" section lists the following headers:

```
access-control-allow-credentials: true
content-length: 30
content-type: application/json
date: Tue, 14 Jul 2026 13:12:26 GMT
server: uvicorn
```

Authorized:

The screenshot shows a Swagger UI interface with a dark theme. The top bar displays "API" and version information "0.1.0" and "OAS 3.1". A modal dialog titled "Available authorizations" is open, showing the following information:

Available authorizations

Scopes are used to grant an application different levels of access to data on behalf of the end user. Each API may declare one or more scopes. API requires the following scopes. Select which ones you want to grant to Swagger UI.

OAuth2PasswordBearer (OAuth2, password)

Authorized

Token URL: `/auth/login`
Flow: `password`
username: `Alina Akhtar`
password: `*****`
Client credentials location: `basic`
client_secret: `*****`

Buttons: `Logout` (green), `Close` (grey).

Adding a task:

→ ↺ 127.0.0.1:8000/docs#/Tasks/create_task_tasks_post

Responses

Curl

```
curl -X 'POST' \
'http://127.0.0.1:8000/tasks/' \
-H 'accept: application/json' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjozLCJleHAiOjE3ODQwMzg0OTR9. -UWEcduzuDml5Fqt4Ugs7HtVlyGzQe9bT70YLzuuCF0' \
-H 'Content-Type: application/json' \
-d '{
  "title": "Task Management API",
  "description": "Description of project",
  "status": "pending",
  "due_date": "2026-08-14"
}'
```

Request URL

```
http://127.0.0.1:8000/tasks/
```

Server response

Code	Details
200	Response body <pre>{ "title": "Task Management API", "description": "Description of project", "status": "pending", "due_date": "2026-08-14", "id": 4, "owner_id": 3 }</pre> Download Response headers <pre>access-control-allow-credentials: true content-length: 133 content-type: application/json date: Tue, 14 Jul 2026 13:14:01 GMT server: uvicorn</pre>

Searching Tasks:

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/tasks/?status=pending' \
-H 'accept: application/json' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjozLCJleHAiOjE3ODQwMzg0OTR9. HF4Zi01OFBUNQxVkuuCAFBn17spoGnPyhQ-Thu2Dw'
```

Request URL

```
http://127.0.0.1:8000/tasks/?status=pending
```

Server response

Code	Details
200	Response body <pre>[{ "title": "string", "description": "string", "status": "pending", "due_date": "2026-07-14", "id": 8, "owner_id": 1 }, { "title": "string", "description": "string", "status": "pending", "due_date": "2026-07-14", "id": 9, "owner_id": 1 }, { "title": "string", "description": "string", "status": "pending", "due_date": "2026-07-14", "id": 10, "owner_id": 1 }]</pre> Download

Getting a single task back:

127.0.0.1:8000/docs/#/Tasks/get_single_task_tasks_task_id_get

Name	Description
task_id * required	
integer	4
(path)	

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/tasks/4' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjozLCJleHAiOjE3ODQwMzg5OTR9. -UMEcduzuDm15Fqt4Ugs7HtVLyGzQe9bT70YLzuuCf0'
```

Request URL

http://127.0.0.1:8000/tasks/4

Server response

Code	Details
200	Response body <pre>{ "title": "Task Management API", "description": "Description of project", "status": "pending", "due_date": "2026-08-14", "id": 4, "owner_id": 3 }</pre> Download

Because not allowed to change someone else task:

127.0.0.1:8000/docs/#/Tasks/get_single_task_tasks_task_id_get

Curl

```
curl -X 'PUT' \
  'http://127.0.0.1:8000/tasks/2' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjozLCJleHAiOjE3ODQwMzg5OTR9. -UMEcduzuDm15Fqt4Ugs7HtVLyGzQe9bT70YLzuuCf0' \
  -d '{
    "title": "string",
    "description": "string",
    "status": "pending",
    "due_date": "2026-07-14"
  }'
```

Request URL

http://127.0.0.1:8000/tasks/2

Server response

Code	Details
404	Error: Not Found Response body <pre>{ "detail": "Task not found" }</pre> Download Response headers <pre>access-control-allow-credentials: true content-length: 27 content-type: application/json date: Tue, 14 Jul 2026 13:15:43 GMT server: uvicorn</pre>

Responses

Code	Description
------	-------------

Links

Task Updation:

The screenshot shows a REST client interface with a dark theme. At the top, the address bar displays the URL `127.0.0.1:8000/docs#/Tasks/get_single_task_tasks_task_id_get`. Below the address bar, there are two buttons: "Execute" (highlighted in blue) and "Clear". The "Responses" section is expanded, showing the following details:

- Curl:** A terminal window showing the curl command for a PUT request to `http://127.0.0.1:8000/tasks/4` with headers for accept, authorization, and content-type, and a JSON body containing task details.
- Request URL:** `http://127.0.0.1:8000/tasks/4`
- Server response:** A table with two columns: "Code" and "Details". The "Code" column shows "200". The "Details" column shows the "Response body" as a JSON object:

```
{  "title": "Task Management API",  "description": "Description of project",  "status": "done",  "due_date": "2026-07-19",  "id": "",  "owner_id": 3}
```

Deletion:

The screenshot shows a REST client interface with a light pink theme. At the top, the address bar displays the URL `127.0.0.1:8000/docs#/Tasks/delete_task_tasks_task_id_delete`. Below the address bar, there is a "Parameters" section with a "Cancel" button. The "Parameters" section contains a table with two columns: "Name" and "Description". The "Name" column contains `task_id` with a red asterisk and the word "required". The "Description" column contains `integer (path)`. Below the table, there is a text input field containing the value `4`. Below the input field, there are two buttons: "Execute" (highlighted in blue) and "Clear". The "Responses" section is expanded, showing the following details:

- Curl:** A terminal window showing the curl command for a DELETE request to `http://127.0.0.1:8000/tasks/4` with headers for accept and authorization.
- Request URL:** `http://127.0.0.1:8000/tasks/4`
- Server response:** A table with two columns: "Code" and "Details". The "Code" column shows "200". The "Details" column shows the "Response body" as a JSON object:

```
{  "message": "Task deleted successfully"}
```

Testing:

```
(venv) (base) PS C:\Users\Alina\Desktop\Internship-Artificizen\Week 2\End-of-Week Capstone Task Management REST API> python -m pytest
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\Alina\Desktop\Internship-Artificizen\Week 2\End-of-Week Capstone Task Management REST API
plugins: anyio-4.14.2
collected 9 items

tests\test_auth.py .... [ 44%]
tests\test_tasks.py ..... [100%]
```